May '85
System Tuning Guide

4. BENCHMARK RESULTS

TP-1

TP1 is a general purpose transaction processing simulation. It consists of four major components.

The requester, written in COBOL and using TPF
The server, written in PL/1
The controller, written in PL/1
The analyzer, written in PL/1

Functionally, the controller is used to terminate a TP1 test run in an orderly fashion by noting the termination of all the requesters and then shutting down all servers. The analyzer generates statistics based on entries in the transaction log.

Statistics include cpu time, elapsed time, average response time, 90% response time, response time profiling and time in queues. The requester and server are general parameter driven programs that can be setup to emulate various environments.

There is no screen I/O or communications. Transactions are started by each requester task after sleeping some random but bounded time.

Input parameters specify the number of reads and the number of updates for each transaction, requester and server loop counts which are used to simulate the processing workload of real transactions and the number of server queues. In addition the sleep time between transactions and the deviation on the sleep time can be specified.

The file I/O is designed to simulate random user access. Each server is driven by a different random key sequence read from a file at the beginning of the test.

TP1 results

For the sake of a standard TP1, the following parameters were constant.

Indexed reads per transaction:	5
Indexed rewrites per transaction:	2
Sequential writes (logging):	1
Requester CPU loop:	5000 (one form)
Server CPU loop:	200
Sleep time:	30 seconds
Sleep deviation:	+/- 5 seconds

XA600

The best results are obtained with 3 duplexed D108 disks, 8 Mb 6 servers 2 requesters with 80 tasks each using adjustments to scheduler info and tuning parameters. The requester priority is higher than the server priority.

Average response time	2.1 sec
90th percentile of response time	3.3 sec
Transaction rate	5.0 /sec

The cpu was approximately 80% utilized
Each disk was approximately 40% utilized

XA400

With 8 Mb and 3 Fuji disks 4 servers 2 requesters with 55 tasks each with scheduler info and tuning parameter changes.

Average response time	1.8 sec
90th percentile of response time	2.7 sec
Transaction rate	3.4 /sec

CPU utilization was approximately 80%
Disk utilization was approximately 25%

FT200

The follwing performance measures are obtained with 8 Mb and 2 Fuji disks with 1 requester with 55 tasks with adjustments in the tuning parameters. The server priovity is set higher than the requestor priority.

Average response time	2.1 sec
90th percentile response time	3.5 sec
Transaction rate	1.7 /sec

The CPU was approximately 85% utilized.
The disks were approximately 18% utilized.

OBSERVATIONS

Task metering had no measurable effect.

Cache utilization significantly impacts performance. This is obviously application dependent, but significant improvements in performance are possible.

Multiple server copies greatly improve performance.

Process priorities are very important. On a FT200, Server priorities were better be higher than requestor. On an XA, the opposite was true. A production application should normally run at higher priority than development. Actual priorities may not be obvious because of the scheduler algorithms.

D103 vs D108 comparisons

Various TP-1 comparisons were made between the 143 MB D103 and the new 448 mb D108 disks. In general, the number of transactions/sec increased by approximately 25% from the D103 to the D108 running the standard TP-1 I/O mix. Response time dropped by as much as 60%.

As an example, here are two different task configurations and the results of the comparison. In each case, the test machine was a FT200 and there were two servers running.

Configuration 1 - 1 server with 20 tasks delaying 2 seconds.
Configuration 2 - 1 server with 60 tasks delaying 15 second.

Transactions / second

		D 103	D 108
Configuration 1	relative file	1.956	2.480
Configuration 1	indexed file	1.600	2.073
Configuration 2	relative file	1.904	2.330
Configuration 2	indexed file	1.500	2.119

Point of Sales Benchmark (POS)

In this benchmark, the customer was primarily concerned with thruput and expandability.

Transaction Profile

One indexed read and rewrite of a 100,000 rec file (512 bytes/rec)
Two lookups and two updates of a 3mb terminal control table
Two sequential writes of 256 byte log entries
50000 character moves
50000 character compares

All code was in PL1 and used the standard Stratus TPF (transaction processing facility). Each transaction, in addition to the above, also executed several thousand additional instructions.

When both XA 600s were used, approximately 65% of the transactions spanned both modules.

Hardware Profile

Two XA 600s each with 8MB duplexed memory and two 448 MB disk drives

Performance

processors	Transactions/sec	avg response time
One XA600	11.6	.96 seconds
Two XA600s	22.8	.97 seconds

Observations

The queueing mechanism in the point-of-sale(POS) benchmark. This benchmark required high transaction through put as well as fast transaction response times. Thus we were pushing the machine to its limits, in order to get the best performance.

Direct queues were used because they are generally faster than server queues and are especially fast between modules. This is because direct queues do not use remote subroutine calls and net_servers in the multimodule case. Whenever possible one way direct queues were used, as they are about twice as fast as two way direct queues.

Rather than have a generic queue for a server class, each server had its own private direct queue. This assured that each message routed by the requester went to a specific server. This provided us with several advantages. First, every server in a server class did NOT wake up when a message was put on the queue, as each had its own private queue it was listening to. Also when we decided to split the database up by having different servers handle different parts of the database, we did NOT have to change one line of code in the servers, only the routing function in the requester. Then automatically the server only received requests in the range that each individual requester was sending to it. This makes the requester code more complicated.

The POS benchmark, parsed the following variables to make this task simpler:

1. a generic queue name (typically 'ps_queue' for Preauth/Standin)

2. the number of this type of queue (typically 6)

3. the maximum number of records in the database (typically 100,000)

By using this scheme we were able to change the entire nature of the benchmark without changing one line of code in the requesters or the servers.

A direct queue can only have one message outstanding per port. To allow each task in a process to simultaneously place a message in the same queue, each task must open the queue individually. Since the database was split amongst multiple servers, each task had to open each one of these queues. For example, if there are 8 tasks in a requester and the database is split in 10 parts, 80 opens are required just for queues by each requester. This uses a large amount of memory, especially when multiplied by the number of requesters running in each module (specifically 5).

To get around this issue, one solution is to design a mechanism to map multiple tasks onto some number of direct queues. There is still the chance of waiting for the direct queue but it is reduced. In the POS benchmark, code written by Larry Sherman was used to do just that. Parameters to the routines specify the number of opens required for each queue, and the code routed requests among the opens and put requests on a waiting list, if all the opens were in use. In the POS benchmark, there were two opens for each direct queue; fewer than the number of tasks being run in the requester.

Order Entry Benchmark

In this benchmark, the customer was concerned with thruput, terminal count and support, and response time.

Transaction Profile

Display and Accept full data entry screen (6 fields)
Two indexed reads and one update of a 100,000 rec file (256 bytes/rec)
One sequential write to a log file.

Code was in COBOL and used the standard TPF environment. Terminals were IBM 3270 type terminals emulated by a hardware simulator.

Hardware Profile

One XA400 with 16 MB duplex memory and two 448 MB disk drives
One XA600 with 16 MB duplex memory and two 448 MB disk drives

Performance

CPU type	% CPU util.	# of terminals	90% resp. time	Avg resp. time	Trans/sec
XA400	75%	144	1.90	1.29	7.6
XA400	98%	192	4.20	2.14	10.1
XA600	75%	220	2.60	1.65	11
XA600	95%	288	4.95	3.00	14.6